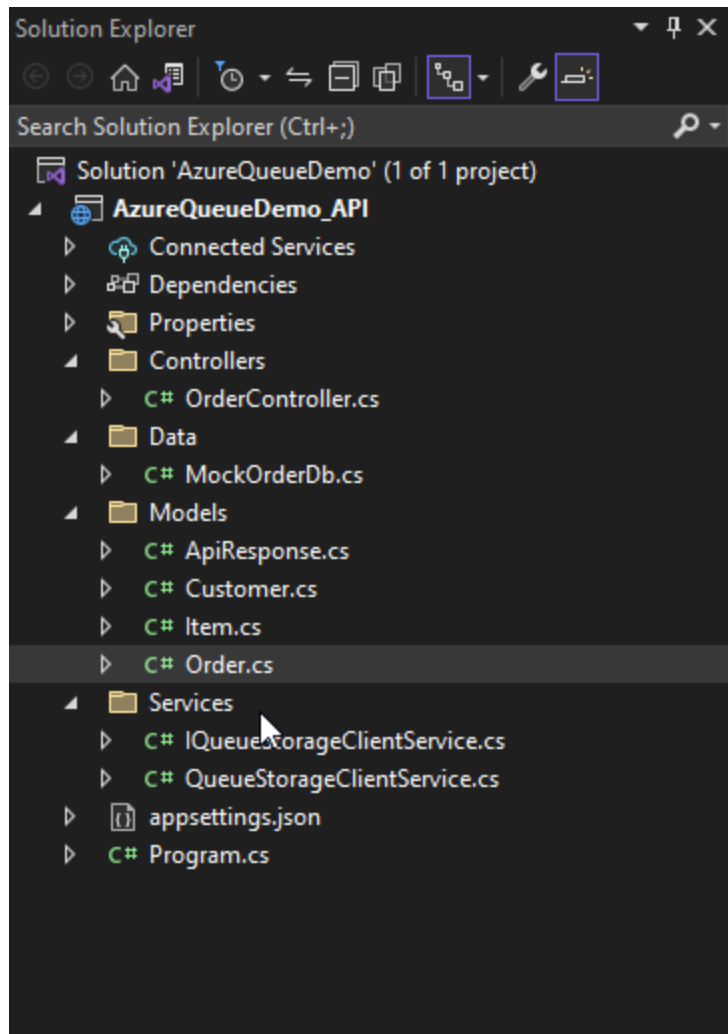
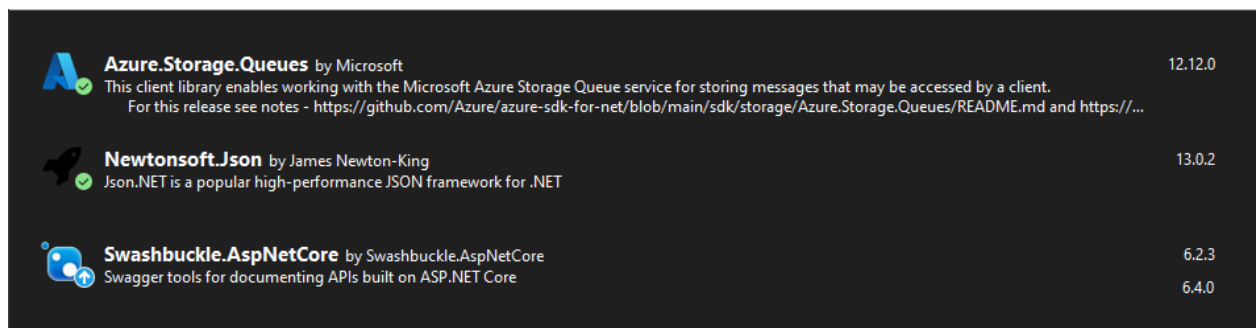


Creating the Solution

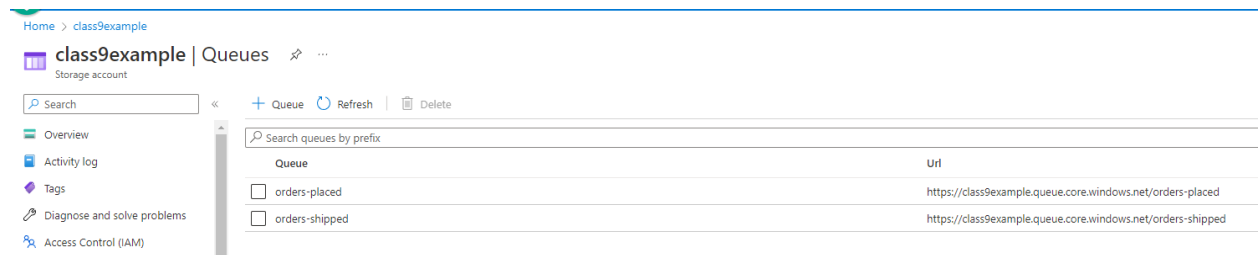
1. Create a dotnet core web API and create the basic project structure. Note that we have not implemented EF core in this solution since we are only interested about azure Queues. This will be the base template to start from and we will add contents as required when moving forward.



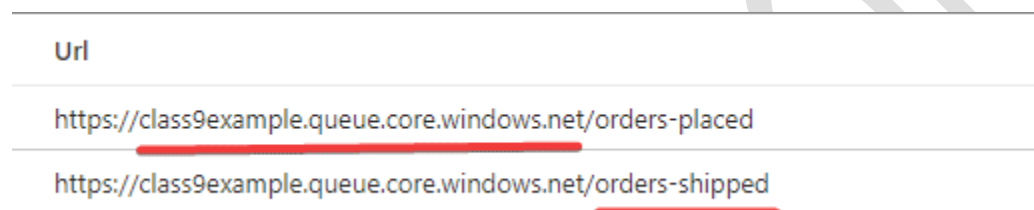
2. Install the required Nuget packages. (Swashbuckle.AspNetCore is installed by default)



3. In your azure portal, create a storage account and in it create 2 queues. Make sure that you follow the practices that we discussed about. The names of the two queues are orders-placed and orders-shipped.

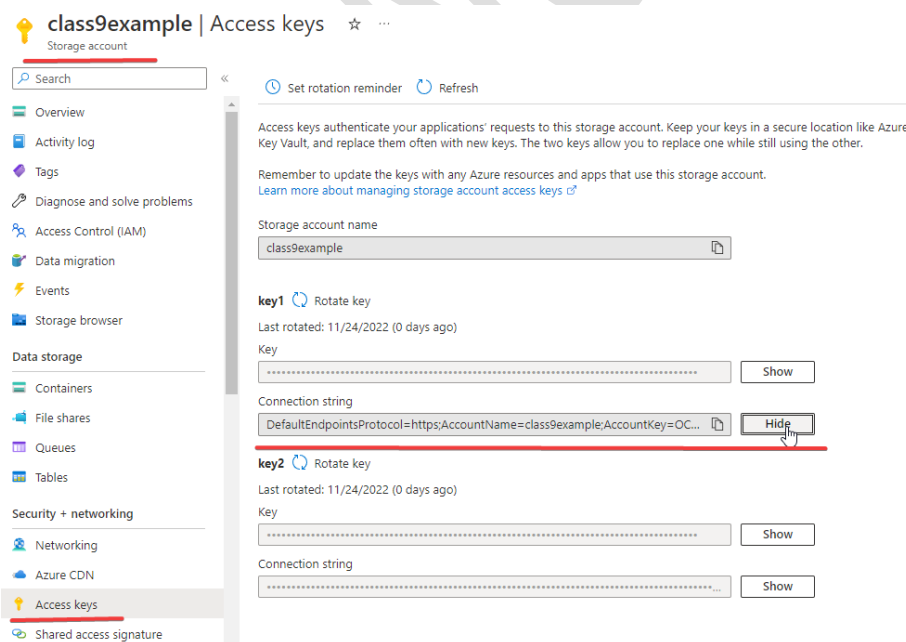


4. Observe the queue URL's it follows the below structure. Keep note of this URL's as we will be needing them in a later stage.



https://<storage-account-url>/<queue-name>

5. At the storage Account level, go to Access Keys and copy one of the keys. We will be needing this to connect to the queue.



DefaultEndpointsProtocol=https;AccountName=class9example;AccountKey=OCGG5SSxy/a8LfkJF3kSJf
nfXhQgqj5pFu8mzP/GQDZhSpHOOZW/T/gq+jAxh2EhhXe/7bFHB580+ASTvwAb+w==;EndpointSuffix=core.windows.net

6. Creating Models

```
public class ApiResponse
{
    public HttpStatusCode StatusCode { get; set; }
    public bool IsSuccess { get; set; }
    public List<string>? ErrorMessages { get; set; }
    public object? Result { get; set; }
}

public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;

    //Many other properties of customer can go here
}

public class Item
{
    public Guid Id { get; set; } = new Guid();
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public decimal UnitPrice { get; set; }
}

public class Order
{
    public Guid Id { get; set; }
    public int CustomerId { get; set; } //FK of Customer
    public Guid ItemId { get; set; } //FK of Item
    public int Units { get; set; }
    public decimal UnitPrice { get; set; }
}
```

7. Creating the Storage Client Service

```
public interface IQueueStorageClientService
{
    QueueClient CreateQueueClient(string conString, string queueName);
}
```

```

using Azure.Storage.Queues;

namespace AzureQueueDemo_API.Services;

public class QueueStorageClientService:IQueueStorageClientService
{
    public QueueClient CreateQueueClient(string conString, string queueName)
    {
        var queueClient = new QueueClient(conString, queueName);
        return queueClient;
    }
}

```

8. AppSettings.json file

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "QueueStorageSettings": {
    "ConnectionString":
    "DefaultEndpointsProtocol=https;AccountName=class9example;AccountKey=OCGG5SSxy/a8Lfk
    JF3kSJfnfXhQgqj5pFu8mzP/GQDZhSpH00ZW/T/gq+jAxh2EhhXe/7bFHB580+ASTvwAb+w==;EndpointSu
    ffix=core.windows.net",
    "OrderQueueName": "orders-placed"
  }
}

```

9. Controller Code

```

using System.Net;
using AzureQueueDemo_API.Data;
using AzureQueueDemo_API.Models;
using AzureQueueDemo_API.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Newtonsoft.Json;

namespace AzureQueueDemo_API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class OrderController : ControllerBase
    {
        private readonly IConfiguration _configuration;
        private readonly IQueueStorageClientService _clientService;
        private readonly ApiResponse _apiResponse;

        public OrderController(IConfiguration configuration,
            IQueueStorageClientService clientService)
        {
            _configuration = configuration;
            _clientService = clientService;
            _apiResponse = new ApiResponse();
        }

        [HttpPost]
        public async Task<ApiResponse> CreateAnOrder([FromBody] Order order)
        {
            var queueClient = _clientService.CreateQueueClient(
                _configuration.GetSection("QueueStorageSettings:ConnectionString").Value,
                _configuration.GetSection("QueueStorageSettings:OrderQueueName").Value);

            //Serializing the order
            var orderDetails = JsonConvert.SerializeObject(order);

            var receiptResponse =
                await queueClient
                    .SendMessageAsync(
                        orderDetails); //Talk about visibility timeout, dequeue
count and time to live overloads in the class

            if (!receiptResponse.GetRawResponse().IsError)
            {
                //Consider as a success
                _apiResponse.ErrorMessages = null;
                _apiResponse.IsSuccess = true;
                _apiResponse.StatusCode = HttpStatusCode.Created;
                _apiResponse.Result = null;
            }
            else
            {
                _apiResponse.ErrorMessages = new List<string>() {
receiptResponse.GetRawResponse().ReasonPhrase };
                _apiResponse.IsSuccess = false;
            }
        }
    }
}

```

10. MockDbCode

```
namespace AzureQueueDemo_API.Data
{
    public static class MockOrderDb
    {
        public static List<Order> ReturnOrderList()
        {
            var orders = new List<Order>()
            {
                new Order() {Id=Guid.NewGuid(),CustomerId = 1,ItemId =
Guid.NewGuid(),Units = 8,UnitPrice = 10},
                new Order() {Id=Guid.NewGuid(),CustomerId = 2,ItemId =
Guid.NewGuid(),Units = 4,UnitPrice = 30},
                new Order() {Id=Guid.NewGuid(),CustomerId = 1,ItemId =
Guid.NewGuid(),Units = 2,UnitPrice = 12},
                new Order() {Id=Guid.NewGuid(),CustomerId = 2,ItemId =
Guid.NewGuid(),Units = 8,UnitPrice = 19},
                new Order() {Id=Guid.NewGuid(),CustomerId = 3,ItemId =
Guid.NewGuid(),Units = 1,UnitPrice = 18},
                new Order() {Id=Guid.NewGuid(),CustomerId = 3,ItemId =
Guid.NewGuid(),Units = 6,UnitPrice = 16},
            };

            return orders;
        }
    }
}
```

11. Programme.cs Code

```
namespace AzureQueueDemo_API
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.

            builder.Services.AddControllers();
            // Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();
            builder.Services.AddScoped<IQueueStorageClientService,
QueueStorageClientService>();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();
        }
    }
}
```

```

        app.UseAuthorization();

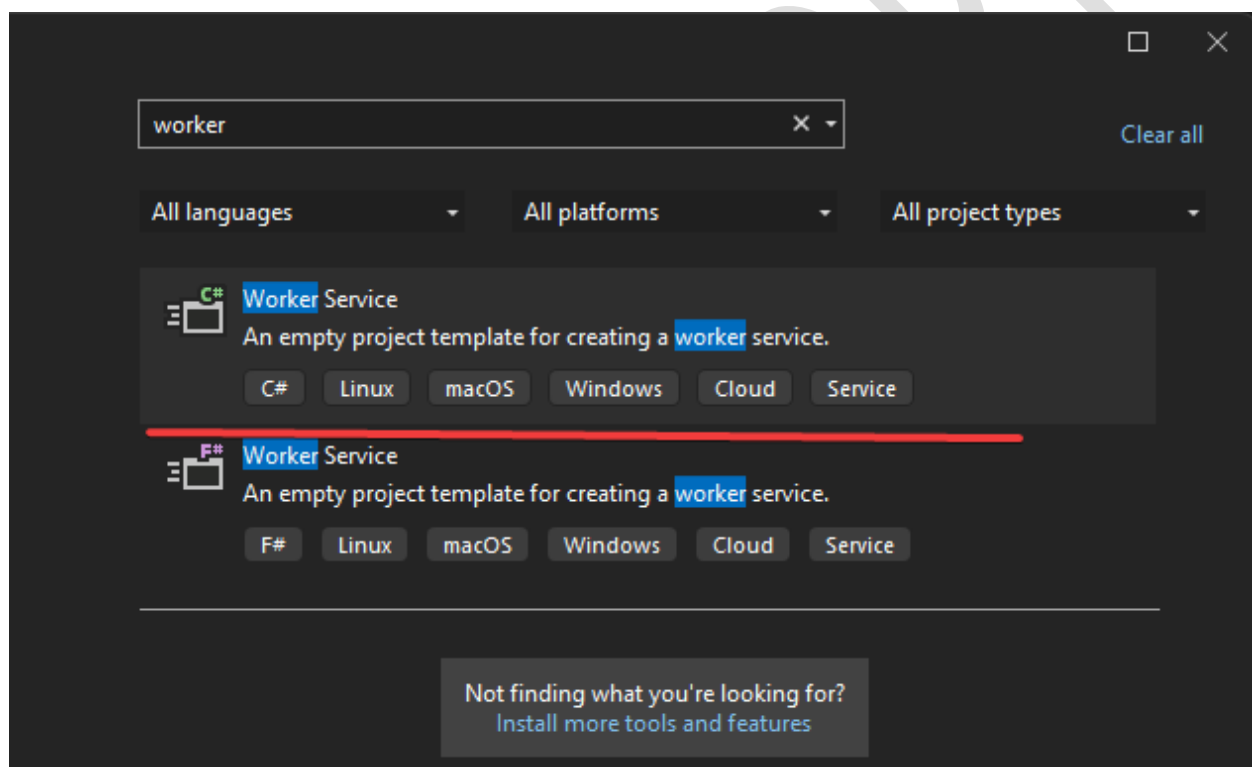
        app.MapControllers();

        app.Run();
    }
}

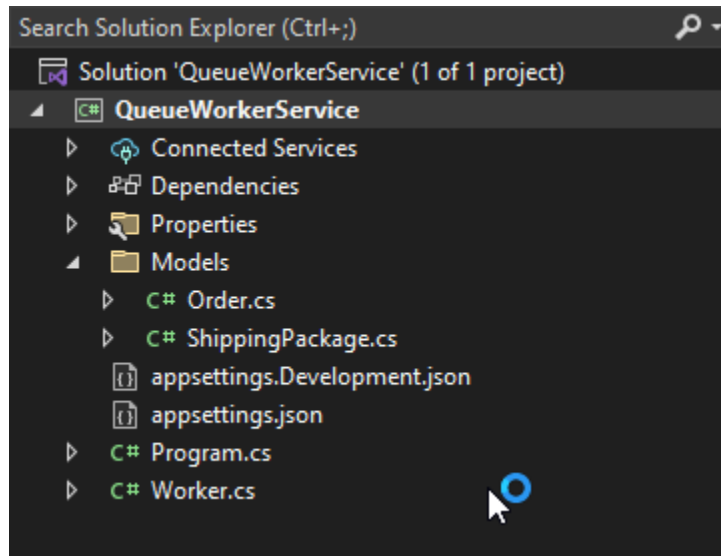
```

Creating The Worker

1. Create a New Worker Service in Visual Studio



2. Folder Structure



3. Models

```
namespace QueueWorkerService.Models;

public class ShippingPackage
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Order? Order { get; set; }
    public string Address { get; set; } = "XYZ";
}

namespace QueueWorkerService.Models;

public class Order
{
    public Guid Id { get; set; }
    public int CustomerId { get; set; }
    public Guid ItemId { get; set; }
    public int Units { get; set; }
    public decimal UnitPrice { get; set; }
}
```

4. Appsettings.json


```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "QueueStorageSettings": {
    "ConnectionString":
"DefaultEndpointsProtocol=https;AccountName=class9example;AccountKey=OCGG5SSxy/a8Lfk
JF3kSJfnfXhQgqj5pFu8mzP/GQDZhSpH00ZW/T/gq+jAxh2EhhXe/7bFHB580+ASTvwAb+w==;EndpointSu
ffix=core.windows.net",
    "OrderQueueName": "orders-placed",
    "ShippingQueueName": "orders-shipped"
  }
}

```

5. Worker Service Code

```

using Azure.Storage.Queues;
using Newtonsoft.Json;
using QueueWorkerService.Models;

namespace QueueWorkerService
{
    public class Worker : BackgroundService
    {
        private readonly ILogger<Worker> _logger;
        private readonly IConfiguration _configuration;

        public Worker(ILogger<Worker> logger, IConfiguration configuration)
        {
            _logger = logger;
            _configuration = configuration;
        }

        protected override async Task ExecuteAsync(CancellationToken stoppingToken)
        {
            #region CreatingQueueClients

            //This is only done for simplicity. Rather this should be done through a
            //service injection
            //These can be singleton instances as well
            var orderClient = new
            QueueClient(_configuration.GetSection("QueueStorageSettings:ConnectionString").Value
            ,
            _configuration.GetSection("QueueStorageSettings:OrderQueueName").Value);

            var shippingClient = new QueueClient(
            _configuration.GetSection("QueueStorageSettings:ConnectionString").Value,
            _configuration.GetSection("QueueStorageSettings:ShippingQueueName").Value);

            #endregion

            while (!stoppingToken.IsCancellationRequested)
            {
                _logger.LogInformation("Worker running at: {time}",
                DateTimeOffset.Now);

                var orderQueueMessage = await
                orderClient.ReceiveMessageAsync(cancellationTokens: stoppingToken);

                if (orderQueueMessage.Value != null) //messages available
                {
                    var order =
                    JsonConvert.DeserializeObject<Order>(orderQueueMessage.Value.MessageText);

                    //Mimic processing and put it into
                    //Creating the shipping Package
                    var shippingPackage = new ShippingPackage()
                    {
                        Order = order!
                    };
                }
            }
        }
    }
}

```

6.

Creating the Azure App Service Instance

1. Go to the Azure portal and create the Azure App Service Instance.

Home >

App Services

Default Directory (skandaudahewaoutlook.onmicrosoft.com)

+ Create ⚙️ Manage view ▾ ↻ Refresh ⬇️ Export to CSV 🔗 Open query | 🏷️ Assign tags ▶ Start ⏪ Restart ☐ Stop 🗑️ Delete

Filter for any field... Subscription equals all Resource group equals all X Location equals all X + Add filter

Name ↑↓	Status ↑↓	Location ↑↓	Pricing Tier
---------	-----------	-------------	--------------



No app services to display

Create, build, deploy, and manage powerful web, mobile, and API apps for your back-end. Build standards-based web apps and APIs using .NET, Java, and more.

[Create app service](#)

[Learn more about App Service](#)

Create Web App ...

[Basics](#) [Deployment](#) [Networking](#) [Monitoring](#) [Tags](#) [Review + create](#)

App Service Web Apps lets you quickly build, deploy, and scale enterprise-grade web, mobile, and API apps running on any platform. Meet rigorous performance, scalability, security and compliance requirements while using a fully managed platform to perform infrastructure maintenance. [Learn more](#) ↗

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ ▼

Resource Group * ⓘ ▼

[Create new](#)

Instance Details

Need a database? [Try the new Web + Database experience.](#) ↗

Name * ✓ .azurewebsites.net

Publish * ☒ Code ☐ Docker Container ☐ Static Web App

Runtime stack * ▼

Operating System * ☐ Linux ☒ Windows

Region * ▼

i Not finding your App Service Plan? Try a different region or select your App Service Environment.

Pricing plans

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#) ↗

Basics Deployment Networking Monitoring Tags Review + create

Summary



Web App
by Microsoft

Free sku

Estimated price - Free

Details

Subscription	60eebb99-a371-42e1-b5e3-0416f364175a
Resource Group	DotNet031
Name	queuestoragedemo
Publish	Code
Runtime stack	.NET 6 (LTS)
Tags	University: Humber

App Service Plan (New)

Name	ASP-DotNet031-a08e
Operating System	Windows
Region	Central US
SKU	Free
ACU	Shared infrastructure
Memory	1 GB memory
Tags	University: Humber

Monitoring

Application Insights	Not enabled
----------------------	-------------

Deployment

2. Now we have created a web App but we have not yet uploaded any code.

Click here to access Application Insights for monitoring and profiling for your app.

Essentials

Resource group [\(move\)](#) : DotNet031

Status : Running

Location [\(move\)](#) : Central US

Subscription [\(move\)](#) : Azure subscription 1

Subscription ID : 60eebb99-a371-42e1-b5e3-0416f364175a

Tags [\(edit\)](#) : University : Humber

URL : <https://questoragedemo.azurewebsites.net>

App Service Plan : ASP-DotNet031-a08e

Operating System : Windows

Health Check : Not Configured

[Properties](#) [Monitoring](#) [Logs](#) [Capabilities](#) [Notifications](#)

Your web app is running and waiting for your content

Your web app is live, but we don't have your content yet. If you've already deployed, it could take up to 5 minutes for your content to show up, so come back soon.

 Supporting Node.js, Java, .NET and more

Haven't deployed yet?

Use the deployment center to publish code or set up continuous deployment.

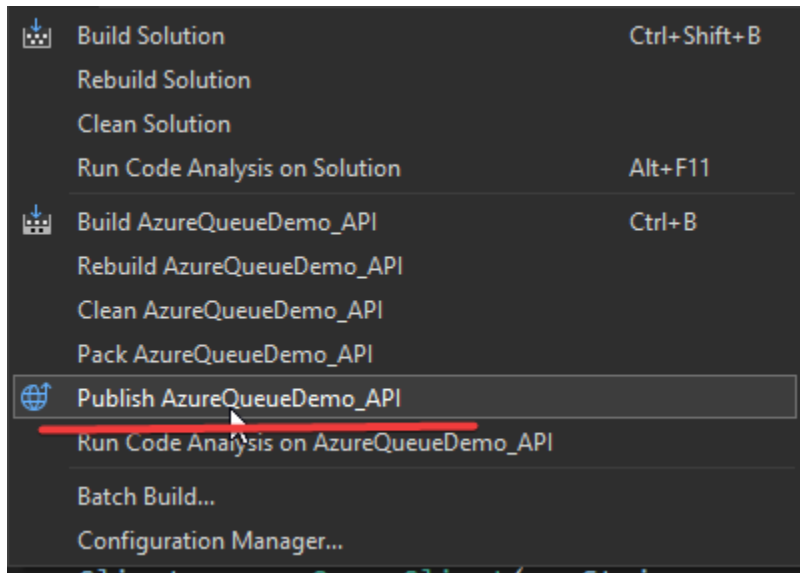
[Deployment center](#)

Starting a new web site?

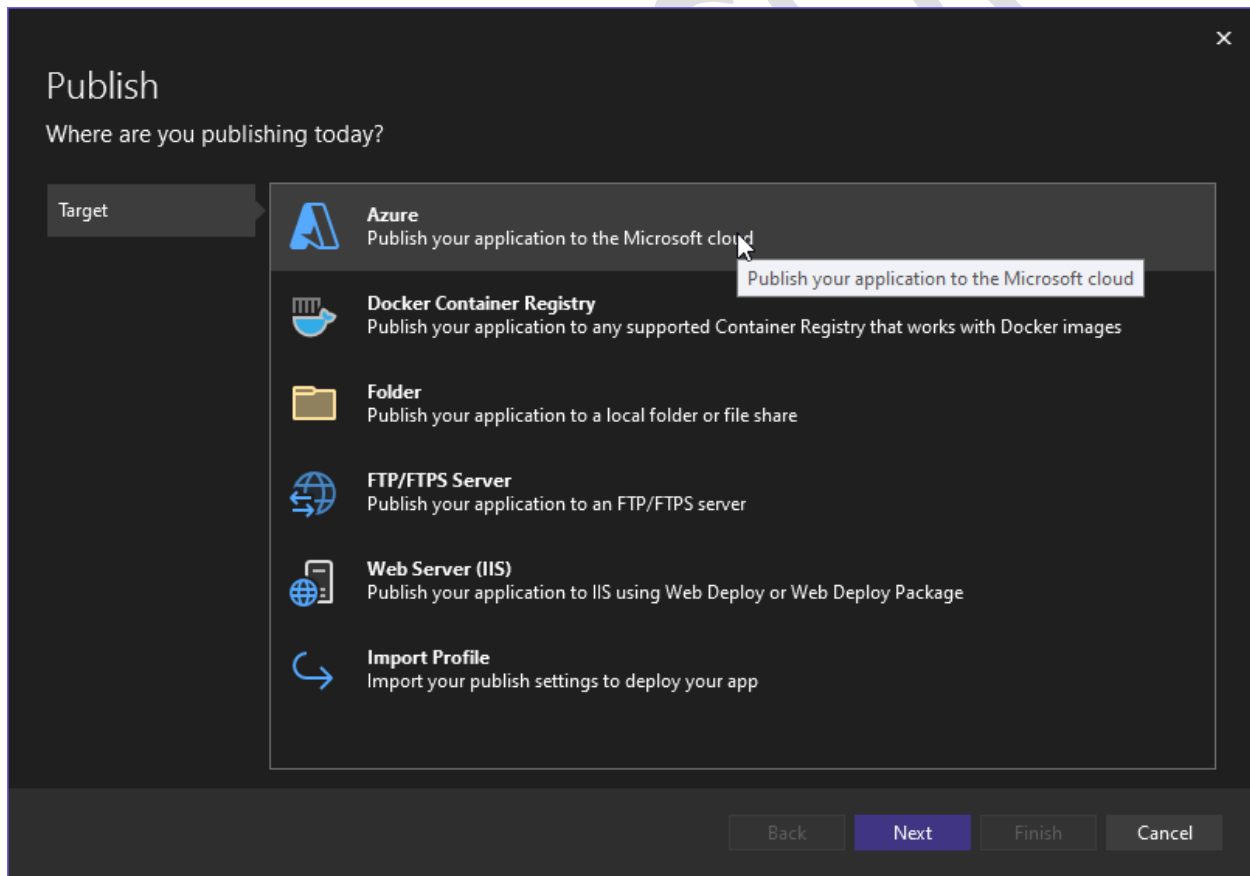
Follow our Quickstart guide to get a web app ready quickly.

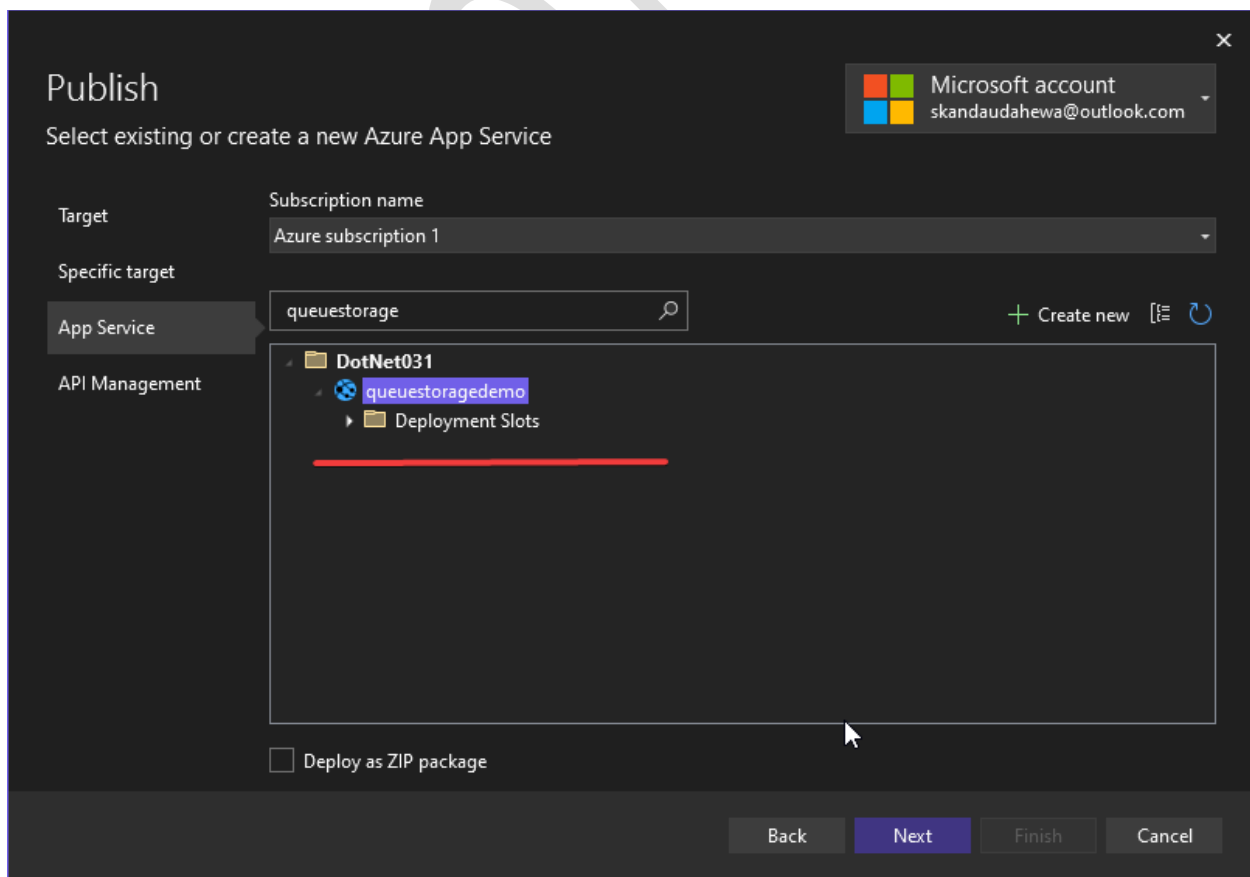
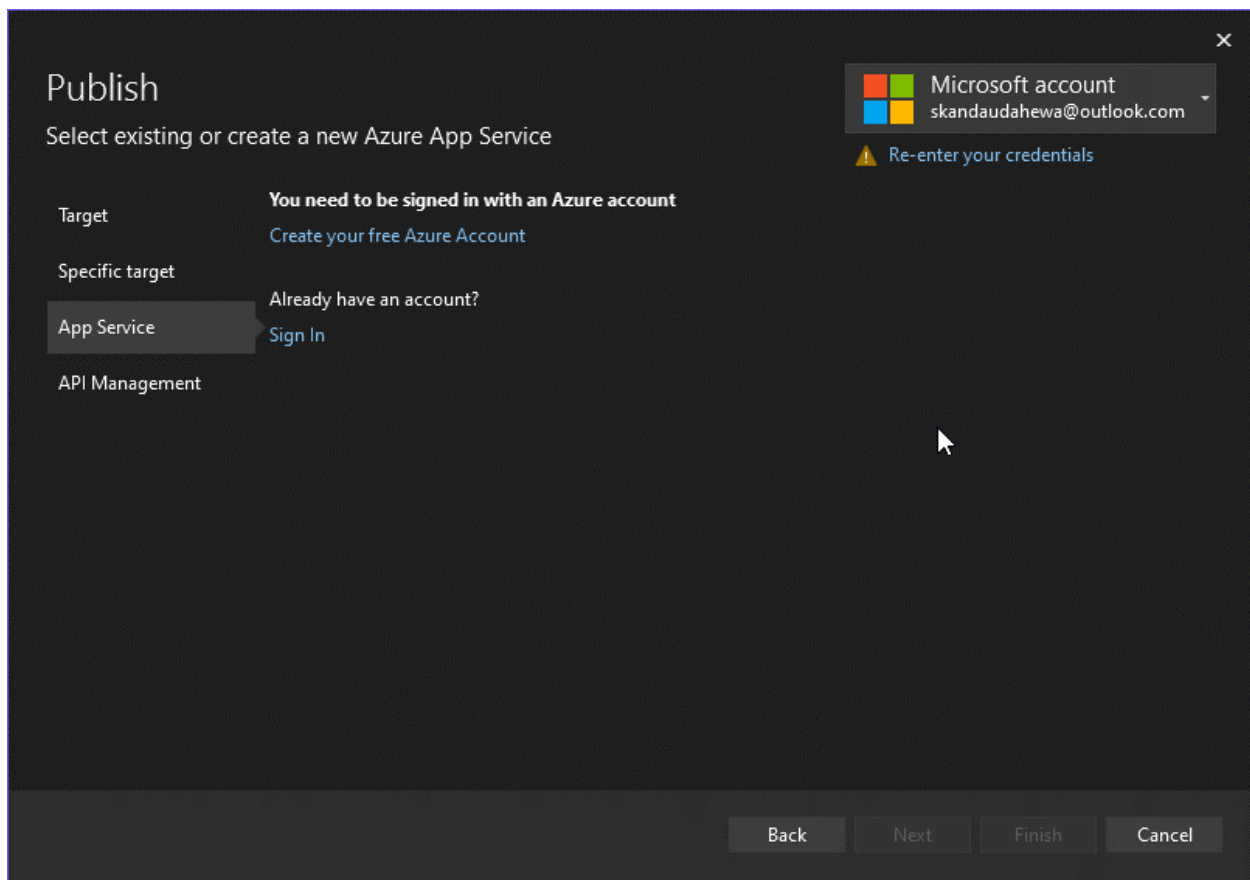
[Quickstart](#)

- Now lets publish the Api that we just created.

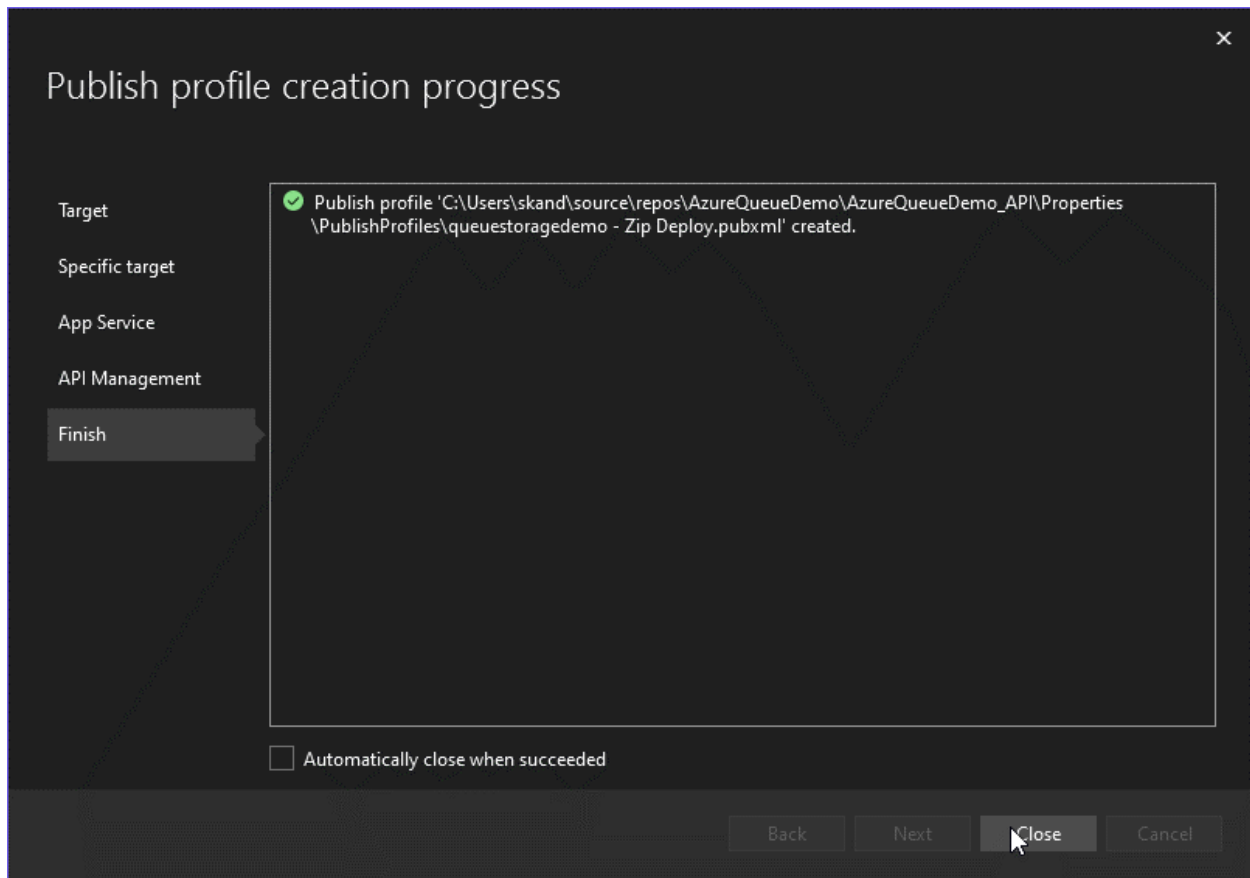


4. You need to connect to your account. (Sign in is needed)





5. Now we see that the publish profile is created. But the code is still not published.



6. Lets publish our code now.

queuestoragedemo - Zip Deploy.pubxml ▾
Azure App Service (Windows) Publish

+ New More actions ▾

Ready to publish.

Settings

Configuration	Release
Target Framework	net6.0
Deployment Mode	Framework-dependent
Target Runtime	Portable

[Show all settings](#)

Hosting

Subscription	60eebb99-a371-42e1-b5e3-0416f364175a
Resource group	DotNet031
Resource name	queuestoragedemo

Site: <https://queuestoragedemo.azurewebsites.net>

Service Dependencies + ↺ ...

Azure API Management ...

Output

Show output from: Build

```
>----- Publish started: Project: AzureQueueDemo_API, Configuration: Release Any CPU -----
AzureQueueDemo_API -> C:\Users\skand\source\repos\AzureQueueDemo\AzureQueueDemo_API\bin\Release\net6.0\AzureQueueDemo_API.dll
AzureQueueDemo_API -> C:\Users\skand\source\repos\AzureQueueDemo\AzureQueueDemo_API\obj\Release\net6.0\PubTmp\Out\
Publishing C:\Users\skand\source\repos\AzureQueueDemo\AzureQueueDemo_API\obj\Release\net6.0\PubTmp\AzureQueueDemo_API-20221124164455021.zip to h
Zip Deployment succeeded.
Web App was published successfully https://queuestoragedemo.azurewebsites.net/
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Publish: 1 succeeded, 0 failed, 0 skipped =====
Installation of Web App Site extension Microsoft.AspNetCore.AzureAppServices.SiteExtension is in progress...
Successfully installed Web App extension Microsoft.AspNetCore.AzureAppServices.SiteExtension
Restarting the Web App...
```

7. After successfully publishing Click on the browse button and type the correct API

AzureQueueDemoAPI20221124165729 ...
App Service

Search « Browse Stop Swap Restart Delete Refresh Download publish profile Reset publish profile Share to mobile Send us your feedback

Overview

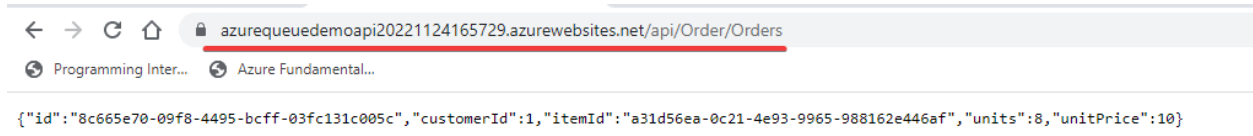
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Microsoft Defender for Cloud
- Events (preview)

Deployment

Click here to access Application Insights for monitoring and profiling for your app.

Essentials

Resource group (move)	: DotNet031	URL	: https://azurequeuedemoapi20221124165729.azurewebsites.net
Status	: Running	App Service Plan	: ASP-DotNet031-a08e (F1: 1)
Location (move)	: Central US	Operating System	: Windows
Subscription (move)	: Azure subscription 1	Health Check	: Not Configured
Subscription ID	: 60eebb99-a371-42e1-b5e3-0416f364175a		
Tags (edit)	:		



8. You should see a json returned as the result.